

Warsaw University of Technology

Institute of IETiSIP

Master's diploma thesis

in the field of study Applied Computer Science

Input-Adaptive Matrix Inference Optimization in Large Language Models through Selective Sparsification

Jan Guziuk
Student record book number 310997

Thesis supervisor
dr hab. inż. Bartosz Sawicki

WARSAW 2025

Input-Adaptive Matrix Inference Optimization in Large Language Models through Selective Sparsification

Abstract. Large language models impose substantial computational costs during inference, with each token requiring matrix multiplications across billions of parameters. This thesis presents input-adaptive selective sparsification, a method that dynamically selects which dimensions of matrix operations to compute based on input-dependent importance scores.

The method was implemented in a custom C++ OpenCL inference engine and evaluated on Mistral 7B. Experimental results demonstrate measurable speedup at moderate sparsity levels with minimal quality degradation across perplexity, token agreement, and factual accuracy metrics. Analysis of the gap between theoretical and observed speedup identifies irregular memory access patterns as the dominant efficiency bottleneck.

The primary contribution is demonstrating that input-adaptive sparsification provides practical inference speedup without permanent model modification, enabling deployment strategies that adapt computation to varying quality requirements and system load.

Keywords: Large Language Models, Inference Optimization, Dynamic Sparsity, GPU Computing, Transformer Architecture

Contents

1. Introduction 4
1.1. Large Language Models and the Scaling Challenge 4
1.2. The Transformer Architecture 5
1.3. Training vs. Inference Workloads 5
1.4. Sparsity in Neural Network Computations 6
2. Input-Adaptive Selective Sparsification 8
2.1. Related Work 8
2.2. Algorithm Formulation 9
2.3. Theoretical Analysis 11
3. Experimental Methodology 13
3.1. Hardware Configuration and Model Selection 13
3.2. Evaluation Framework 15
4. Implementation 21
4.1. System Design 21
4.2. Weight Representation and Preprocessing 22
4.3. Sparsification Kernels 24
4.4. Sparse Multiplication Kernel Optimization 26
4.5. Software Architecture 27
5. Experimental Results and Analysis 29
5.1. Computational Efficiency 30
5.2. Selection Mechanism Overhead 32
5.3. Output Quality Degradation 34
5.4. Baseline Comparison 37
5.5. Synthesis of Experimental Findings 38
5.6. Quality-Efficiency Trade-off 39
5.7. Comparison to Related Work 40
5.8. Analysis of the Speedup Gap 41
6. Conclusion 43
References 44

1. Introduction

1.1. Large Language Models and the Scaling Challenge

Large language models (LLMs) have become dominant in modern artificial intelligence, achieving strong performance in natural language understanding and reasoning. This progress has been enabled primarily by the transformer architecture [1] and by scaling of model size, training data, and computational resources.

The evolution of model sizes illustrates this trend: GPT-2 (2019) [2] contained 1.5 billion parameters, GPT-3 (2020) [3] scaled to 175 billion parameters, and subsequent models such as PaLM [4], Llama 2 [5], and Mistral [6] have continued this trajectory. While scaling has led to consistent performance improvements, it has also resulted in rapidly increasing computational demands.

Scaling Laws. Empirical studies have shown that language model loss decreases according to approximate power-law relationships with respect to the number of parameters N, the size of the training dataset D, and the available compute budget C [7], [8]. When other factors are held constant, this behavior can be expressed as:

$$L(N, D, C) \approx L_\infty + AN^{-\alpha} + BD^{-\beta} + GC^{-\gamma}, \tag{1}$$

where L_∞ denotes the irreducible loss and α, β, and γ are empirically determined scaling exponents.

These scaling laws explain why increasing model size has been a successful strategy for improving performance. However, they also imply a direct increase in computational cost: larger models require more resources not only during training but also during inference.

Training models at the scale of systems such as ChatGPT requires thousands of hours of GPU computation [3]. While this cost is incurred only once, deployed models must handle potentially millions of inference requests. As a result, **inference-time computation becomes a primary bottleneck**, since each request involves a full forward pass through billions of parameters. Reducing inference cost is a challenge for the practical deployment of LLM's.

This thesis addresses this challenge by investigating input-adaptive selective sparsification of matrix computations in order to reduce inference-time computational cost.

1.2. The Transformer Architecture

The transformer architecture [1] is composed of two main parts: a self-attention mechanism and a feedforward network.

Self-Attention. The self-attention mechanism enables each token to attend to all other tokens in the sequence. For an input sequence represented as matrix $X \in \mathbb{R}^{n \times d}$, three projections are computed:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \tag{2}$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ are learned weight matrices. The attention output is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{3}$$

Multi-head attention extends this by computing h parallel attention operations with separate projections, then concatenating and projecting the results through $W_O \in \mathbb{R}^{hd_k \times d}$.

Feedforward Network. Each transformer layer also contains a position-wise feedforward network:

$$\text{FFN}(x) = \sigma(xW_{\text{up}}) \cdot (xW_{\text{gate}}) \cdot W_{\text{down}} \tag{4}$$

where $W_{\text{up}}, W_{\text{gate}} \in \mathbb{R}^{d \times d_{ff}}$ and $W_{\text{down}} \in \mathbb{R}^{d_{ff} \times d}$. Typically $d_{ff} = 4d$, making feedforward layers the largest source of parameters and computation.

Computational Profile. For a model like Mistral 7B [6] with $d = 4096$, $d_{ff} = 14336$, and 32 layers, each transformer layer contains seven weight matrices:

- Attention: W_Q, W_K, W_V, W_O (each 4096×4096)
- Feedforward: $W_{\text{gate}}, W_{\text{up}}$ (4096×14336), W_{down} (14336×4096)

During inference, processing a single token requires 224 matrix-vector multiplications (7 matrices × 32 layers), each involving thousands of multiply-accumulate operations. This computational burden motivates the search for efficient approximations.

1.3. Training vs. Inference Workloads

Training and inference impose fundamentally different computational requirements, justifying distinct optimization strategies.

Training. During training, model parameters are updated via gradient descent. Each forward pass is followed by a backward pass computing gradients for all parameters, roughly doubling computational cost compared to inference. Training requires large batches for statistical stability and stores intermediate activations for backpropagation,

demanding substantial memory. Training occurs once (or infrequently for fine-tuning), making total cost finite and predictable.

Inference. During inference, parameters are fixed and only forward passes execute. In autoregressive generation, tokens are produced sequentially: each new token requires recomputing attention over all previous tokens, creating per-token latency that directly impacts user experience. Unlike training's batch-oriented throughput, inference often optimizes for single-request latency.

Inference cost scales with deployment: a model serving millions of users accumulates inference costs that can far exceed one-time training costs. This Economic considerations motivates inference-specific optimizations that may be unsuitable for training, such as the input-adaptive method proposed in this thesis.

Optimization Implications. Static optimization techniques—pruning, quantization, knowledge distillation—modify the model once and apply uniformly to all inputs. These methods reduce both training and inference costs but sacrifice model capacity permanently.

Dynamic optimization techniques make runtime decisions during inference, adapting computation to each input without modifying weights. This approach preserves full model capacity while reducing average inference cost, but requires runtime overhead for decision-making. Input-adaptive selective sparsification, the focus of this thesis, belongs to this category.

1.4. Sparsity in Neural Network Computations

The key observation enabling selective sparsification is that neural network computations often exhibit redundancy: not all operations contribute equally to the output.

Definition. A matrix is **sparse** when a substantial fraction of its elements are zero or near-zero. Sparsity can be [9]:

- **Structured**: zeros appear in predictable patterns (blocks, bands, entire rows/-columns)
- **Unstructured**: zeros distributed irregularly throughout the matrix

Sources of Sparsity in LLMs. Sparsity arises naturally in neural networks through several mechanisms [10]:

1. **Activation sparsity**: ReLU and similar activations produce zero outputs for negative inputs, creating sparse activation vectors
2. **Attention sparsity**: softmax attention concentrates probability mass on few tokens, making most attention weights near-zero
3. **Weight magnitude distribution**: trained weights often follow heavy-tailed distributions, with many small-magnitude values contributing minimally to outputs

Exploiting Sparsity. If sparsity can be identified efficiently, computation can be reduced by skipping operations involving zero or negligible values. For a matrix-vector product $y = Wx$ where $x \in \mathbb{R}^n$:

$$y = \sum_{i=1}^{n} W_{:,i} \cdot x_i \tag{5}$$

If only $k < n$ dimensions of x have significant magnitude, computing only those k terms reduces complexity from $O(mn)$ to $O(mk)$. The challenge lies in identifying which dimensions are significant—ideally without examining all of them, which would negate computational savings.

Static vs. Dynamic Sparsification. Static sparsification (pruning) permanently removes weights below a threshold, creating fixed sparse matrices [11]. This approach:

- Reduces model size and memory bandwidth requirements
- Applies the same sparse pattern to all inputs
- Requires retraining or fine-tuning to recover accuracy
- Cannot adapt to input-specific importance patterns

Dynamic sparsification determines at runtime which computations to perform based on the current input [12]. This approach:

- Preserves full model capacity (no weights removed)
- Adapts sparsity patterns to each input
- Incurs runtime overhead for importance estimation
- Enables input-dependent quality-efficiency trade-offs

This thesis develops an input-adaptive selective sparsification method that combines importance estimation with dynamic threshold selection, aiming to achieve inference speedup while preserving model quality through input-dependent computation allocation.

2. Input-Adaptive Selective Sparsification

This section reviews existing optimization techniques for neural network inference, positions the proposed method within this landscape, and presents the theoretical framework for input-adaptive selective sparsification.

2.1. Related Work

Optimization techniques for neural network inference can be categorized into **static methods**, which permanently modify model structure or parameters, and **dynamic methods**, which make runtime decisions during inference.

Static Compression Methods.

- *Pruning* removes parameters or connections that contribute little to model performance [11]. Approaches range from unstructured weight pruning (setting individual weights to zero based on magnitude) to structured pruning (removing entire neurons, attention heads, or layers) [9]. While pruning can achieve high compression rates, it permanently reduces model capacity and typically requires fine-tuning to recover accuracy.
- *Quantization* reduces numerical precision of parameters, mapping 32-bit values to lower precision formats like FP16, INT8 or INT 4) [13]. This reduces memory footprint and enables faster computation on specialized hardware, but introduces quantization error that accumulates through layers.
- *Low-rank approximation* decomposes weight matrices using techniques such as SVD, replacing a matrix $W \in \mathbb{R}^{m\times n}$ with lower-rank factors $W \approx UV^T$ where $U \in \mathbb{R}^{m\times k}$ and $V \in \mathbb{R}^{n\times k}$ for $k \ll \min(m,n)$ [14]. This reduces parameters but introduces approximation error proportional to discarded singular values.
  Knowledge distillation trains a smaller model to imitate the output distributions of a larger one, transferring its capabilities to a more efficient architecture [15].

All static methods share a limitation: optimizations are fixed at deployment and cannot adapt to varying input complexity. A statically pruned model applies the same sparse structure whether processing a simple or complex input.

Dynamic Inference Methods. Dynamic methods adapt computation at runtime based on input characteristics. *Early exit* mechanisms allow models to terminate at intermediate layers when prediction confidence is high [16]. *Mixture-of-Experts* (MoE) architectures use learned gating mechanisms to direct inputs toward specialized expert subnetworks [17]. *Adaptive computation time* methods learn to allocate variable computation per input [18].

These approaches share the principle of **conditional computation**—adapting computational effort to input requirements rather than applying uniform computation. Input adaptive selective sparsification belongs to this category but operates at a finer granularity:

selecting which dimensions of matrix operations to compute rather than which layers or experts to activate.

Positioning of This Work. Input-adaptive selective sparsification differs from other existing approaches:

- **vs. Static Pruning**: Weights remain unchanged; the full model capacity is preserved. Selection occurs at the operation level, not the parameter level.
- **vs. Quantization**: Does not reduce numerical precision; instead reduces the number of operations by computing only important contributions.
- **vs. Early Exit / MoE**: Operates within each layer's matrix multiplications rather than at the layer or subnetwork level, providing finer-grained control.

The method is also **complementary** to static techniques: a quantized model can still benefit from selective sparsification, combining reduced precision with adaptive computation.

2.2. Algorithm Formulation

The input-adaptive selective sparsification algorithm operates in four stages for each matrix-vector multiplication: (1) computing importance scores for all input dimensions, (2) selecting a dynamic threshold based on the target sparsity, (3) constructing the set of dimensions to compute, and (4) performing the sparse multiplication. This subsection formalizes each stage.

2.2.1. Problem Statement

Consider a matrix-vector multiplication $y = Wx$ where $W \in \mathbb{R}^{m \times n}$ is a weight matrix and $x \in \mathbb{R}^n$ is an input activation vector. The output can be written as a sum over input dimensions:

$$y = \sum_{i=1}^{n} W_{:,i} \cdot x_i \tag{6}$$

where $W_{:,i}$ denotes the i-th column of W. Standard computation requires $O(mn)$ operations.

The key observation is that for any specific input x, not all dimensions contribute equally. Dimensions where $|x_i|$ is small or where $W_{:,i}$ has low magnitude contribute little to the output. Input-adaptive selective sparsification identifies and computes only the significant terms.

2.2.2. Importance Scoring

To determine which dimensions to compute, the algorithm assigns an importance score to each input dimension. Let $p \in \mathbb{R}^n$ be a vector of **probe values** computed during

preprocessing:

$$p_i = \frac{1}{m} \sum_{j=1}^{m} |W_{j,i}| \tag{7}$$

representing the mean absolute weight magnitude in column i. This captures the typical contribution of dimension i across all outputs.

For each inference step with input x, the importance score for dimension i is:

$$I_i = \alpha \cdot |x_i \cdot p_i| \tag{8}$$

where α is a scaling constant ensuring numerical stability. The product captures both the current activation magnitude and the dimension's typical influence on outputs.

2.2.3. Dynamic Threshold Selection

Given a target computation fraction $\sigma \in (0,1]$, the algorithm selects a threshold τ such that approximately $\sigma \cdot n$ dimensions are retained. The threshold is computed as the $(1-\sigma)$-quantile of the importance distribution:

$$\tau = \text{quantile}(I, 1-\sigma) \tag{9}$$

2.2.4. Sparse Computation

The set of selected dimensions is:

$$\mathscr{D} = \{i : I_i \geq \tau\} \tag{10}$$

The approximate output is computed using only selected dimensions:

$$\tilde{y} = \sum_{i \in \mathscr{D}} W_{:,i} \cdot x_i \tag{11}$$

The weight matrix W remains unchanged—sparsification occurs at the level of which operations to execute, not which parameters to retain. This means:

- Full model capacity is preserved
- Different inputs activate different dimension subsets
- The sparsity pattern $\mathscr{D}$ varies per token and per layer

When $\sigma = 1.0$, all dimensions are selected and $\tilde{y} = y$ exactly.

2.3. Theoretical Analysis

This subsection analyzes the computational and quality implications of selective sparsification. We derive the theoretical speedup as a function of the sparsity parameter σ, establish bounds on approximation error, and characterize key properties that distinguish this approach from static methods.

2.3.1. Computational Complexity

Baseline. Standard matrix-vector multiplication requires $O(mn)$ operations.

Sparsification Overhead. The selection mechanism introduces:

- Importance computation: $O(n)$ — element-wise product and absolute value
- Threshold selection: $O(n)$ — binary search with parallel counting
- Total overhead: $O(n)$ per matrix operation

Sparse Multiplication. With $|\mathscr{D}| \approx \sigma n$ dimensions selected:

$$\text{Cost}_{\text{sparse}} = O(n) + O(\sigma mn) = O(n(1+\sigma m)) \tag{12}$$

Speedup. The theoretical speedup factor is:

$$\text{Speedup} = \frac{mn}{n(1+\sigma m)} = \frac{m}{1+\sigma m} \tag{13}$$

For transformer dimensions where $m \gg 1$ (e.g., $m = 4096$), the overhead term becomes negligible and speedup approaches $1/\sigma$:

- At $\sigma = 0.5$: theoretical $2\times$ speedup
- At $\sigma = 0.25$: theoretical $4\times$ speedup

2.3.2. Error Analysis

The approximation error introduced by selective sparsification is:

$$\Delta y = y - \tilde{y} = \sum_{i \notin \mathscr{D}} W_{:,i} \cdot x_i \tag{14}$$

This represents the contribution of excluded dimensions. The error magnitude depends on the importance of excluded dimensions and the weight matrix structure.

Error Bound. Let $I_{\text{total}} = \sum_{i=1}^{n} I_i$ be the total importance. By construction, excluded dimensions have importance below the threshold:

$$I_{\text{excluded}} = \sum_{i \notin \mathscr{D}} I_i \leq (1-\sigma) \cdot I_{\text{total}} \tag{15}$$

Since importance scores approximate contribution magnitudes through probe values:

$$\|\Delta y\|_2 \lesssim \frac{1-\sigma}{\alpha} \cdot I_{\text{total}} \tag{16}$$

Error scales linearly with the exclusion rate $(1-\sigma)$, providing a quality-efficiency trade-off.

Input-Adaptive Behavior. Error depends on the specific input's importance distribution:

- **Concentrated importance**: If most importance is in few dimensions, high sparsity achieves low error
- **Distributed importance**: Uniform importance distribution leads to higher error at the same sparsity

This adaptivity contrasts with static pruning, which removes the same weights regardless of input characteristics.

2.3.3. Key Properties

Capacity Preservation. Since weights are never modified, the full model remains available. At $\sigma = 1.0$, computation is identical to dense baseline.

Layer Independence. Each layer's matrices are sparsified independently using layer-specific probe vectors, allowing different layers to focus computation on different representation aspects.

Interpolation. The parameter σ provides continuous control over the quality-efficiency trade-off, from full computation ($\sigma = 1.0$) to aggressive sparsification ($\sigma \to 0$).

3. Experimental Methodology

This section describes the experimental design, evaluation metrics, datasets, and measurement procedures used to assess input-adaptive selective sparsification. The methodology addresses four primary research questions: (1) Does selective sparsification achieve measurable speedup? (2) What is the computational cost of the selection mechanism? (3) How does output quality degrade with sparsity (4) How does the selective sparsification compare to base model?

3.1. Hardware Configuration and Model Selection

3.1.1. Hardware Platform

All experiments are conducted on a consumer-grade GPU:

- **GPU**: AMD Radeon RX 7900 XT
- **Architecture**: RDNA 3
- **VRAM**: 20 GB GDDR6
- **Compute Units**: 84 CUs (5376 stream processors)

Software Stack. The implementation uses a custom C++/OpenCL inference engine (described in Section 4) running on:

- **Operating System**: Ubuntu 22.04 LTS
- **OpenCL Runtime**: AMD ROCm 5.7
- **Compiler**: Clang 16.0
- **Precision**: FP16 weight storage

3.1.2. Model Selection Criteria

The selection of a suitable large language model (LLM) is guided by the following criteria:

- **Open availability of weights and code** – models must be licensed to enable direct access to parameters and matrix operations, essential for implementing selective sparsification.
- **Architectural diversity** – including dense and mixture-of-experts (MoE) variants, to examine the sparsification mechanism under different computational regimes.
- **Parameter scale** – a range of models from 7B to 70B parameters was considered, enabling stepwise scaling of experiments.
- **Hardware feasibility** – models must be runnable on the AMD RX 7900 XT (20 GB VRAM), either natively or with quantization and offloading.
- **Community and tooling support** – preference for models with Hugging Face integration [19] and open-source ecosystems.

3.1.3. Model Selection

Based on these criteria, eight candidate models were evaluated, summarized in Table 3.1. Models ranged from 7B to 70B parameters, including both dense architectures (Mistral 7B [6], Falcon 40B [20], Gemma 3 [21]) and Mixture-of-Experts variants (DeepSeek R1 [22], Qwen 3 [23]).

Model	Parameters	Architecture	Context Length	Feasibility (20 GB VRAM)
Mistral 7B [6]	7B	Dense, decoder-only	32k	Native
Falcon 40B [20]	40B	Dense, decoder-only	8k	Quantized + offload
Gemma 3 [21]	~7B	Dense	32k	Native
Qwen 3 [23]	14B / 32B	Dense	128k	Quantized
DeepSeek R1 [22]	~37B active	Sparse MoE	128k	Quantized + offload
Salamandra 7B	7B	Dense	32k	Native

Table 3.1. Candidate LLMs considered for selective sparsification experiments.

Mistral 7B [6] was selected as the primary model for the following reasons:

For the experimental evaluation, **Mistral 7B** [6] was selected as the primary model. This decision is motivated by the following factors:

1. **Hardware compatibility**: The 7B parameter count ensures the model runs natively on the AMD RX 7900 XT without quantization or CPU offloading, enabling accurate performance measurements.
2. **Rapid iteration**: The modest size of 7B parameters allows for quick experimentation during implementation development and debugging.
3. **Representative architecture**: Mistral 7B is a dense transformer using modern techniques (RoPE embeddings [24], SwiGLU activation [25], grouped-query attention [26]), making it representative of current LLM design.
4. **Open availability**: Full model weights and architecture details are publicly available under an Apache 2.0 license.

Model Architecture Details. Mistral 7B is a decoder-only transformer with 32 identical layers. Key architectural parameters:

- **Hidden dimension**: 4096
- **Intermediate FFN dimension**: 14336
- **Attention heads**: 32 (query), 8 (key/value, grouped-query attention)
- **Vocabulary size**: 32000 tokens
- **Sequence length**: 32768 tokens (with RoPE)
- **Total parameters**: Approximately 7.24 billion

Each transformer layer contains 7 weight matrices subject to selective sparsification:

- Attention: $W_Q, W_K, W_V, W_O \in \mathbb{R}^{4096\times4096}$
- Feedforward: $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{4096\times14336}$, $W_{\text{down}} \in \mathbb{R}^{14336\times4096}$

3.2. Evaluation Framework

This subsection defines the metrics and dataset used to assess computational efficiency, output quality, memory characteristics, and sparsity patterns.

3.2.1. Computational Efficiency Metrics

Three primary metrics quantify the inference speedup achieved by selective sparsification:

End-to-end latency (ms/token). The total time required to generate a single token during autoregressive decoding, measured from the start of the forward pass through all 32 transformer layers until the output logits are computed. This metric captures the user-perceived performance and includes all computational operations: matrix multiplications, normalization, activation functions, and selection overhead.

Latency is measured using OpenCL event profiling with microsecond precision. Lower latency indicates better performance.

Throughput (tokens/s). The number of tokens generated per second, computed as:

$$\text{Throughput} = \frac{1000}{\text{Latency (ms)}} \tag{17}$$

Throughput directly translates to serving capacity in production deployments: higher throughput allows a single GPU to handle more concurrent requests. This metric is relevant for assessing deployment economics.

Speedup factor. The ratio of baseline latency ($\sigma = 1.0$, full computation) to sparse latency at a given sparsity level:

$$\text{Speedup}(\sigma) = \frac{\text{Latency}_{\sigma=1.0}}{\text{Latency}_{\sigma}} \tag{18}$$

Speedup quantifies the relative performance gain compared to dense inference. A speedup of 2.0× indicates the sparse model runs twice as fast as the baseline. This metric enables comparison to theoretical predictions from Section 2.3, where speedup was predicted as $1/\sigma$.

3.2.2. Selection Overhead Metrics

To quantify the computational cost of the runtime selection mechanism, we decompose each sparse matrix operation into four kernels and measure their individual execution times.

Kernel Timing Breakdown. Four kernels constitute the selective sparsification pipeline for each matrix operation:

1. **Threshold selection kernel**: Computes importance scores $s_i = |x_i| \cdot p_i$ for all input dimensions and determines threshold τ. Expected complexity: $O(n)$.
2. **Dispatch list construction kernel**: Builds sparse dispatch list $\mathscr{D}$ containing indices and values of selected dimensions using atomic operations. Expected complexity: $O(n)$.
3. **Sparse multiplication kernel**: Performs sparse matrix-vector multiplication on selected dimensions only, computing $y_{\text{partial}} = \sum_{i \in \mathscr{D}} x_i W_{:,i}$. Expected complexity: $O(\sigma nm)$.
4. **Result integration kernel**: Combines partial results across workgroups and accumulation positions to produce final output. Expected complexity: $O(m)$.

Each kernel's execution time is measured in microseconds using OpenCL event profiling. The absolute time for each kernel and its percentage contribution to total operation time are recorded.

Selection Overhead Definition. The **selection overhead** is defined as the combined time of all operations except the actual sparse multiplication:

$$t_{\text{overhead}} = t_{\text{threshold}} + t_{\text{dispatch}} + t_{\text{integrate}} \tag{19}$$

The overhead percentage relative to total operation time is:

$$\text{Overhead \%} = \frac{t_{\text{overhead}}}{t_{\text{total}}} \times 100 = \frac{t_{\text{overhead}}}{t_{\text{threshold}} + t_{\text{dispatch}} + t_{\text{multiply}} + t_{\text{integrate}}} \times 100 \tag{20}$$

This metric quantifies the cost of the adaptive selection mechanism relative to the computation it enables. Low overhead percentage indicates efficient selection; high overhead suggests the fixed $O(n)$ costs dominate the $O(\sigma nm)$ multiplication savings.

Per-Element Overhead. To contextualize the overhead, we compute the per-element selection cost:

$$\text{Per-element overhead} = \frac{t_{\text{overhead}}}{n} \quad \text{(nanoseconds per dimension)} \tag{21}$$

This metric indicates how much computational cost is incurred per input dimension to determine whether it should be selected. Lower per-element overhead suggests the selection mechanism scales efficiently with problem size.

3.2.3. Output Quality Metrics

Output quality degradation is assessed using three complementary metrics that capture statistical prediction quality, distributional shifts, and token-level consistency:

Perplexity (PPL). Perplexity shows how well a probabilistic model predicts a token sequence by indicating its uncertainty about the next token [27]. It is the standard metric for evaluating language models and provides a direct measure of predictive quality [28]. Given a test sequence $\mathbf{x} = (x_1, x_2, \ldots, x_n)$, perplexity is defined as:

$$\mathrm{PPL}(\mathbf{x}) = \exp\left(-\frac{1}{n}\sum_{i=1}^{n}\log P(x_i \mid x_{<i})\right) \tag{22}$$

where $P(x_i \mid x_{<i})$ is the conditional probability the model assigns to token x_i given all preceding tokens.

Lower perplexity means the model gives higher probability to the observed sequence, indicating better prediction quality. Perplexity increases monotonically with quality degradation, making it an ideal metric for quantifying the impact of selective sparsification. In LLM compression literature, a perplexity increase of 5% or less is typically considered acceptable quality degradation.

KL Divergence. Kullback-Leibler divergence [29] measures the distributional difference between baseline and sparse model output distributions, quantifying how much the probability distribution over the vocabulary changes due to selective sparsification. For output distributions P (baseline, $\sigma = 1.0$) and Q (sparse, $\sigma < 1.0$) over the vocabulary V, KL divergence is:

$$D_{\mathrm{KL}}(P\|Q) = \sum_{i=1}^{V} P(i)\log\frac{P(i)}{Q(i)} \tag{23}$$

where $V = 32000$ is the vocabulary size for Mistral 7B.

KL divergence quantifies information loss when using Q to approximate P. A value of 0 indicates identical distributions, while larger values indicate greater divergence. Unlike perplexity, which measures absolute prediction quality on held-out data, KL divergence specifically measures how selective sparsification alters the shape of the output distribution. This metric is sensitive to changes throughout the entire distribution, not just the top-ranked tokens, enabling detection of degradation patterns such as probability mass redistribution or confidence reduction.

KL divergence is computed at each token position during inference on the evaluation dataset, then averaged across all positions to produce a single aggregate metric per sparsity level.

Token Agreement Rate. Token agreement measures the percentage of token positions where the baseline and sparse models select the same top-1 token under greedy decoding (argmax selection) [15], [30]:

$$\text{Agreement} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{1}[\arg\max_j P_i(j) = \arg\max_j Q_i(j)] \times 100\% \tag{24}$$

where N is the number of token positions evaluated, P_i and Q_i are the baseline and sparse distributions at position i, and $\mathbb{1}[\cdot]$ is the indicator function.

High token agreement (>95%) indicates that despite distributional differences measured by perplexity and KL divergence, the model still makes the same greedy predictions. This metric is particularly relevant for deployment scenarios using greedy or low-temperature sampling, where only the top-ranked token matters. Token agreement provides a measure of prediction consistency that complements the statistical measures provided by perplexity and KL divergence.

Factual Correctness (MMLU Accuracy). While perplexity, KL divergence, and token agreement measure statistical quality, distributional shifts, and prediction consistency, they do not directly assess whether the model produces factually correct outputs. A model may maintain high token agreement while selecting incorrect answers, or exhibit low perplexity while producing false statements. To accpount for this limitation, we test factual correctness using the Massive Multitask Language Understanding (MMLU) benchmark [31].

MMLU is a standardized multiple-choice benchmark covering 57 subjects including mathematics, history, science, law, and social sciences. Each question presents a prompt and four answer choices (A, B, C, D), with one correct answer. The benchmark tests both factual knowledge recall (e.g., historical dates, scientific constants) and reasoning ability (e.g., mathematical problem-solving, logical deduction).

MMLU Evaluation Protocol. For each question, the model is prompted with the question text and four labeled answer choices. The predicted answer is determined by computing the conditional probability of each choice token (A, B, C, D) following the question prompt:

$$\hat{a} = \arg\max_{c \in \{A,B,C,D\}} P(c \mid \text{question, choices}) \tag{25}$$

where $P(c \mid \text{question, choices})$ is the probability the model assigns to token c given the full question context.

Accuracy is computed as:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of questions}} \times 100\% \tag{26}$$

MMLU accuracy provides a direct measure of whether selective sparsification preserves the model's ability to produce factually correct outputs. Unlike perplexity (which measures confidence in any prediction, correct or incorrect) or token agreement (which measures whether predictions match baseline, without validating correctness), MMLU accuracy validates that the model continues to select the right answer from multiple possibilities.

A model with degraded MMLU accuracy despite stable perplexity indicates that sparsification has compromised the model's factual knowledge or reasoning ability, even if statistical quality metrics remain acceptable. Conversely, a model maintaining high

MMLU accuracy despite increased perplexity demonstrates that the model still produces correct outputs even when less confident.

3.2.4. Evaluation Dataset and Sampling

Quality metrics are evaluated on two datasets: WikiText-2 [32] for statistical quality assessment and MMLU [31] for factual correctness validation.

WikiText-2: Statistical Quality Metrics. Perplexity, KL divergence, and token agreement are measured on 1,000 consecutive tokens sampled from the WikiText-2 validation set [32]. This subset is obtained by taking the first 1,000 tokens from the validation split, maintaining context continuity while ensuring computational tractability.

With N=1,000 samples and observed standard deviation of approximately 0.9, the standard error of perplexity measurements is 0.028 (0.19% relative error), providing sufficient precision to detect quality degradation effects in the 2-10% range. Evaluation time is approximately 10 minutes for all ten sparsity levels (approximately 1 minute per sparsity level).

MMLU: Factual Correctness Validation. MMLU accuracy is evaluated on 200 questions sampled from four subjects [31]:

- **Elementary Mathematics** (50 questions): Tests numerical reasoning, arithmetic operations, fractions, percentages, and basic algebra. Evaluates the model's ability to perform calculations and solve quantitative problems.
- **Astronomy** (50 questions): Tests factual knowledge about celestial bodies, planetary systems, astronomical phenomena, and space science. Evaluates scientific knowledge recall.
- **Computer Science** (50 questions): Tests understanding of algorithms, data structures, programming concepts, and computational theory. Evaluates technical reasoning and domain-specific knowledge.
- **Anatomy** (50 questions): Tests knowledge of human body systems, organs, biological processes, and medical terminology. Evaluates life sciences knowledge retention.

These four subjects provide coverage across quantitative reasoning (mathematics), natural sciences (astronomy, anatomy), and technical domains (computer science), ensuring the benchmark assesses multiple dimensions of factual knowledge and reasoning capability.

3.2.5. Deterministic Execution

To ensure reproducibility:

- **Fixed random seed**: All experiments use random seed 42 for token sampling
- **Deterministic kernels**: OpenCL kernels use deterministic algorithms (no random number generation)

- **Greedy sampling**: Token generation uses argmax selection (no stochastic sampling during measurement)
- **Fixed input sequences**: WikiText-2 validation split and prompt set are identical across all runs, the same is true for MMLU.

3.2.6. Baseline Comparison Protocol

All quality metrics are evaluated by comparing sparse configurations ($\sigma < 1.0$) against the baseline ($\sigma = 1.0$):

Primary Baseline: $\sigma = 1.0$ (Sparse Pipeline, Full Selection). The sparse implementation with all dimensions selected serves as the primary baseline. This isolates the effect of dimension selection from implementation artifacts, as both baseline and sparse configurations use the identical code path (threshold selection, dispatch construction, sparse multiplication, integration).

Any quality differences between $\sigma = 1.0$ and $\sigma < 1.0$ can be attributed solely to dimension exclusion rather than implementation differences.

Validation Baseline: Dense Mistral 7B. To validate implementation correctness, a subset of experiments compare $\sigma = 1.0$ against the standard Hugging Face Mistral 7B implementation [19] using dense PyTorch operations. This validation check confirms that the sparse pipeline with full selection produces outputs equivalent to dense computation within numerical precision.

4. Implementation

This section describes the implementation of the input-adaptive selective sparsification mechanism for large language model inference. The implementation is designed to validate the theoretical framework presented in Section 2 and to enable systematic experimentation with different sparsity levels on transformer-based models. The system is implemented in C++ with GPU acceleration via OpenCL, targeting AMD RDNA3 architecture.

4.1. System Design

The implementation is built using C++ as the primary language, with GPU computation offloaded to OpenCL kernels. This choice reflects design priorities specific to the research objectives of this thesis.

First, C++ provides direct control over memory layout and allocation, which is required for implementing custom sparse computation patterns. Unlike high-level frameworks such as PyTorch or TensorFlow, which abstract memory management and operator dispatch, C++ allows explicit control over when and how matrix operations are executed. This level of control is necessary because input-adaptive selective sparsification requires runtime decision-making about which operations to perform—a pattern not natively supported by existing deep learning frameworks.

Second, OpenCL enables portable GPU acceleration across different hardware vendors. While CUDA is more widely used in deep learning research, it is restricted to NVIDIA hardware. OpenCL provides access to AMD GPUs, which were available for this research, while maintaining the ability to port the implementation to other platforms if needed. The OpenCL kernels implement all performance-critical operations: importance score computation, threshold selection, dispatch list construction, and sparse matrix multiplication.

Third, the decision to implement a custom inference engine rather than modifying an existing framework ensures that the sparsification mechanism is integrated into the execution path. In a typical framework, matrix multiplications are dispatched to optimized BLAS libraries that assume dense computation. Intercepting these calls to inject sparsification logic would introduce overhead and complexity.

4.2. Weight Representation and Preprocessing

This subsection describes the packed weight format and preprocessing pipeline that form the foundation for sparse computation.

Packed FP16 Format and Structured Accumulation. All weight matrices in the transformer model are stored in a packed 16-bit floating-point format that combines the numerical value with a 4-bit position index. This representation enables efficient sparse computation while maintaining memory footprint.

Encoding scheme. Each weight value w_{ij} is encoded as a 16-bit quantity where 12 bits store the numerical value and 4 bits store a position index $p_{ij} \in [0, 15]$ indicating the location in the output accumulation array. The encoding uses the FP16 mantissa bits:

```
FP16 format: [sign:1][exponent:5][mantissa:10]
Packed format: [sign:1][exponent:5][value:6][position:4]
```

The lower 4 mantissa bits are replaced with the position index. This causes approximately 0.1% precision loss relative to standard FP16 but eliminates the need for a separate integer index array, reducing memory requirements by 50% compared to storing values and indices separately.

Memory layout. Weight matrices are stored in row-major order with all values packed sequentially. For a 4096×4096 weight matrix, this requires $4096^2 \times 2 = 32$ MB of GPU memory per matrix. The entire Mistral 7B model (256 weight matrices across 32 layers) requires approximately 14 GB in this format, matching the expected size for 7 billion FP16 parameters with minimal overhead.

Weight Preprocessing Pipeline. Before inference can begin, all model weights must be converted from their original storage format (typically PyTorch tensors or Hugging Face safetensors) into the packed binary representation. This conversion is performed by a Python preprocessing script that operates on GPU to accelerate the process.

The preprocessing pipeline consists of four stages:

Stage 1: Model Loading. The script loads the pretrained model weights from the safetensors format, which stores weights as memory-mapped numpy arrays. For Mistral 7B, this involves loading 32 transformer layers, each containing 8 weight matrices: four for attention (W_Q, W_K, W_V, W_O), three for the feedforward network ($W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$), and one for layer normalization.

Stage 2: Probe Vector Extraction. For each weight matrix $W \in \mathbb{R}^{m \times n}$, the preprocessing script computes a probe vector $p \in \mathbb{R}^n$ that captures the statistical importance of each input dimension. The probe value for dimension i is computed as:

$$p_i = \frac{1}{m} \sum_{j=0}^{m-1} |W_{j,i}| \tag{27}$$

This represents the average magnitude of weights in column i, providing an estimate of how strongly input dimension i typically influences the output. During inference, the probe vector is multiplied element-wise with the input activation vector to compute importance scores, as described in Section 2.2.

Stage 3: Position Assignment and Packing. Each weight value is assigned to an accumulation position based on its location in the output dimension. For matrices with output dimension $m = 4096$ and 16 positions, output index j is assigned to position $p_j = j \bmod 16$. This ensures an even distribution of outputs across positions. The packing operation combines the FP16 weight value with the position index into a single 16-bit word.

Stage 4: Binary Serialization. The packed weights, probe vectors, and statistics are written to binary files for efficient loading during inference. Each matrix generates three files:

- Binary dump of packed weights (FP16)
- Binary dump of statistics (mean absolute values per input dimension)
- Metadata file containing dimensions and probe values in human-readable text format

This preprocessing step is performed once per model and takes approximately 5 minutes for Mistral 7B. The resulting binary files total approximately 14 GB, matching the expected size for 7 billion FP16 parameters.

Relationship to Selective Sparsification. The packed weight representation is designed specifically to support input-adaptive selective sparsification. Three properties of this format are:

1. Unchanged weights. The weights themselves are stored in their original learned values (converted to FP16 for memory efficiency). Unlike static pruning approaches that set weights to zero, this representation preserves the full model capacity, enabling dynamic selection at inference time.

2. Probe vectors for importance scoring. The probe vector p is stored alongside the weights and uploaded to GPU memory. During inference, computing importance scores $I_i = \alpha|x_i \cdot p_i|$ requires only element-wise multiplication and absolute value operations, which can be performed in $O(n)$ time.

3. Structured accumulation for sparse computation. The position indices embedded in the packed format enable parallel sparse matrix multiplication. When processing a sparse dispatch list, multiple threads can accumulate results into different positions without synchronization, and the final integration step combines positions into the output vector.

This weight representation forms the foundation for all subsequent sparsification operations.

4.3. Sparsification Kernels

This subsection provides a description of the four-stage algorithm that implements input-adaptive selective sparsification for each matrix multiplication operation during transformer inference.

Importance Score Computation. The importance scoring mechanism, formalized in Section 2.2, computes an importance score for each input dimension based on the current activation values and precomputed probe vectors. This section describes the GPU implementation of this computation.

The importance scores are computed as part of the threshold selection kernel rather than as a separate pass. This fusion eliminates an additional kernel launch and memory round-trip. The kernel launches 256 threads, with each thread responsible for processing 16 consecutive dimensions of the input vector, achieving n-way parallelism for $n = 4096$ dimensions.

Scaling factor selection. Typical importance values computed as $|x_i \cdot p_i|$ fall in the range $[10^{-6}, 10^{-3}]$. The implementation applies a scaling constant $\alpha = 10^5$, shifting the working range to approximately $[0.1, 100]$. This avoids precision issues with very small values during threshold comparison, lessening the error while computing the cutoff point.

Integration with threshold selection. After computing importance scores, each thread holds 16 local importance values. These are immediately used in the subsequent min/max reduction and binary search phases, maintaining data locality and minimizing memory traffic. The importance scores are not written back to global memory during normal operation, existing only in thread-local registers throughout the threshold selection process.

Dynamic Threshold Selection. The threshold selection algorithm, formalized in Section 2.2, determines a threshold τ such that approximately $\sigma \cdot n$ dimensions exceed it. This section describes the GPU implementation using binary search.

The implementation avoids explicit sorting, which would require $O(n \log n)$ time and introduce significant overhead. Instead, a binary search strategy with parallel counting computes an approximate quantile in $O(n)$ time per iteration with high accuracy.

Binary search implementation. The search iteratively refines a threshold $\tau = (\tau_{\min} + \tau_{\max})/2$. In each iteration, threads count dimensions exceeding the current threshold and aggregate counts via wavefront reduction. Based on whether the total exceeds the target $\sigma \cdot n$, the bounds are updated.

Convergence behavior. The loop terminates when: (1) exact match between count and target, (2) numerical convergence of bounds (difference $< 10^{-5}$), or (3) close approximation (count difference < 3). **Threshold storage.** The final threshold τ is written to a single-element GPU buffer. This value is read by the dispatch list construction kernel without CPU involvement, maintaining all selection logic on the GPU and avoiding PCIe transfer latency.

Sparse Dispatch List Construction. Once the threshold τ has been determined, the algorithm constructs a dispatch list containing all input dimensions whose importance exceeds the threshold. The dispatch list is a compact representation of the sparse computation pattern for this specific input.

The dispatch list is implemented as an array of paired values, where each entry stores the input activation value x_i and a pointer (encoded as an integer offset) to the corresponding column of the weight matrix in GPU memory. This representation enables the sparse matrix multiplication kernel to directly access the required weight columns without indirection.

The dispatch list construction kernel launches 64 threads, with each thread processing a chunk of $n/64 = 64$ dimensions. For each dimension in its chunk, the thread recomputes the importance score and compares it against the threshold. If the importance exceeds the threshold, the thread appends an entry to the dispatch list using an atomic counter to reserve a unique index. This guarantees thread-safe construction without locks or barriers.

The dispatch list buffer is preallocated to the maximum possible size (n entries) to avoid dynamic allocation during inference. In practice, for $\sigma = 0.5$, the list contains approximately $0.5 \cdot 4096 = 2048$ entries, consuming $2048 \times 8 = 16$ KB of GPU memory per matrix multiplication.

Sparse Matrix Multiplication. The final stage performs the actual matrix-vector multiplication, computing only the contributions from the selected dimensions. Given a weight matrix $W \in \mathbb{R}^{m \times n}$, input vector $x \in \mathbb{R}^n$, and dispatch list $D = \{(x_i, W_{:,i}) : I_i \geq \tau\}$, the output is:

$$y = \sum_{(x_i, W_{:,i}) \in D} W_{:,i} \cdot x_i \tag{28}$$

This computation has complexity $O(|D| \cdot m) = O(\sigma nm)$ compared to $O(nm)$ for dense multiplication.

The sparse multiplication is implemented in two GPU kernels: one for parallel accumulation and one for final reduction.

Parallel Sparse Accumulation Kernel. This kernel is launched with 64 workgroups of 256 threads each. The dispatch list is partitioned across workgroups, with each workgroup processing approximately $|D|/64$ entries. Each thread is responsible for computing one output column.

The kernel operates in three nested loops:

1. Outer loop: Process dispatch list in cache-sized chunks (256 entries)
2. Middle loop: Cooperatively load chunk into local memory
3. Inner loop: Process each cached entry, accumulating into 16 separate positions

For each dispatch entry $(x_i, W_{:,i})$, the thread loads the weight value from column $W_{:,i}$ at its assigned row, decodes the position index, multiplies by the input value x_i, and accumulates into the appropriate position. The 16 positions enable lock-free parallel

accumulation: different workgroups write to different group indices in the output buffer without synchronization.

LDS Caching. Dispatch list entries are fetched from global memory in blocks of 256 and cached in local data share (LDS). This amortizes the latency of global memory access: each entry is loaded once and used by all 256 threads in the workgroup, providing a 256× effective bandwidth multiplication. The cache size (256 entries × 8 bytes = 2 KB) is chosen to balance LDS capacity with temporal locality.

Integration Kernel. After parallel accumulation, a second kernel performs tree reduction to combine the results across the 64 workgroups and 16 positions into a single output value per dimension.

Performance characteristics. The sparse matrix multiplication could achieves its theoretical complexity of $O(\sigma nm)$ as analyzed in Section 2.3. In practice, the actual runtime is dictated by memory bandwidth rather than computation throughput.

The key bottleneck is global memory access for weight columns. Despite LDS caching of the dispatch list, each weight value must be fetched from global memory exactly once. With 16-bit weights and σnm total elements accessed, the theoretical minimum memory traffic is:

$$\text{Traffic} = 2 \cdot \sigma nm \text{ bytes} \tag{29}$$

4.4. Sparse Multiplication Kernel Optimization

The sparse multiplication kernel described in Section 4.3 represents the largest computational cost of selective sparsification. This subsection describes an optimization targeting memory access patterns and presents experimental tuning of the cache size parameter.

Baseline Implementation. The baseline implementation performs direct global memory access for each dispatch list entry. Within each workgroup, every thread independently reads dispatch entries from global VRAM, incurring substantial latency per access. When multiple threads within a workgroup process the same dispatch entries, this approach generates redundant memory transactions.

LDS Cache Optimization. The optimized implementation leverages the Local Data Share (LDS), a software-managed cache local to each Compute Unit offering 1-2 cycle access latency. Rather than independent global memory access, threads cooperatively load dispatch entries into LDS in chunks: a single coalesced load brings each entry into the shared cache, after which all threads access the cached values with minimal latency. This cooperative loading strategy amortizes the global memory latency across all threads rather than incurring it redundantly for each for T threads accessing shared data.

The optimization introduces a tunable parameter: the dispatch cache size, controlling how many entries are cached per iteration. This parameter governs a tradeoff between cache effectiveness and GPU occupancy.

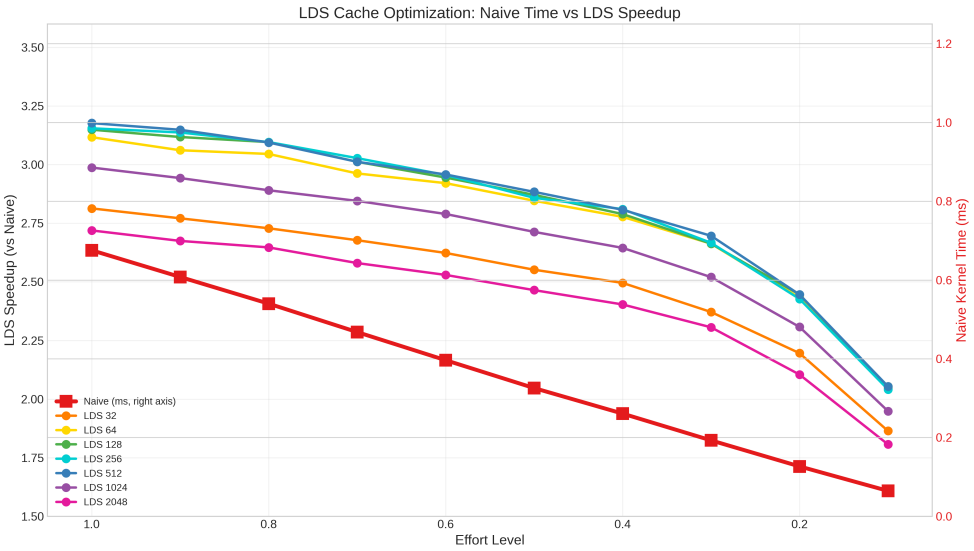


Figure 4.1. LDS speedup by cache size (left axis) and naive kernel time (right axis, red) across effort levels.

The relative ordering of cache configurations remains stable regardless of workload scale, indicating that the memory access patterns and data reuse characteristics are proportionally preserved as σ decreases. Speedup degrades gradually at lower effort levels—from 3.2× at $\sigma = 1.0$ to approximately 2.0× at $\sigma = 0.1$—reflecting the increasing ratio of fixed overhead to computation rather than any change in cache behavior. Cache sizes between 128 and 512 entries perform similarly, with 512 having a marginal advantage. Below 128 entries, performance degrades slightly (2.8× at 32 entries). Above 512 entries, performance drops more noticeably—1024 and 2048 entry configurations fall to 3.0× and 2.7× respectively, likely due to reduced workgroup occupancy. Based on these results, the implementation uses a dispatch cache size of 512 entries.

4.5. Software Architecture

The selective sparsification system is implemented as a custom C++ inference engine with GPU acceleration via OpenCL. This section describes the software architecture used to organize the implementation.

Modular component structure. The implementation is organized into a hierarchy of classes that mirror the transformer architecture:

- **GPU Context Manager:** Provides a high-level interface to OpenCL functionality, abstracting device management, kernel compilation, and buffer allocation. Maintains

a single context and command queue, ensuring operation serialization for accurate timing.

- **Binary Weight Loader:** Handles deserialization of preprocessed weight files and upload to GPU memory. Uses memory-mapped I/O for efficient transfer of the 14 GB weight dataset, completing loading in approximately 8 seconds.
- **Sparsification Module:** Encapsulates the four-stage selective sparsification algorithm as a reusable component that can be applied to any matrix multiplication in the model.
- **Transformer Components:** Separate classes implement attention (Q, K, V, O projections), feedforward networks (gate, up, down projections), and complete transformer layers (combining attention, FFN, and normalization).
- **Model Assembly:** The top-level class assembles 32 transformer layers and manages embeddings, output projections, and autoregressive generation with KV-caching.

5. Experimental Results and Analysis

This section presents the experimental evaluation of input-adaptive selective sparsification applied to the Mistral 7B model and analyses the implications of the results. Subsections 5.1–5.4 present experimental measurements addressing four research questions:

1. **Computational Efficiency:** Does selective sparsification achieve measurable speedup, and how does latency scale with sparsity parameter σ?
2. **Selection Overhead:** What is the computational cost of the runtime selection mechanism relative to sparse multiplication itself?
3. **Output Quality:** How does model output quality degrade as σ decreases, and what is the practical lower bound for acceptable performance?
4. **Comparison to Base Mistral 7B:** How do measured metrics compare to the baseline Mistral 7B implementation?

Subsections 5.5–5.8 synthesise these findings, characterise the quality-efficiency trade-off, compare against related work, and analyse the gap between theoretical and observed speedup.

5.1. Computational Efficiency

This subsection evaluates inference speedup achieved by selective sparsification using end-to-end latency, throughput, and speedup metrics defined in Section 3.2.1, measured following the OpenCL profiling and statistical aggregation procedures described in Section 3.4.

Latency and Throughput Scaling. Figure 5.1 presents end-to-end latency and throughput measurements across all sparsity levels, demonstrating the expected inverse relationship.

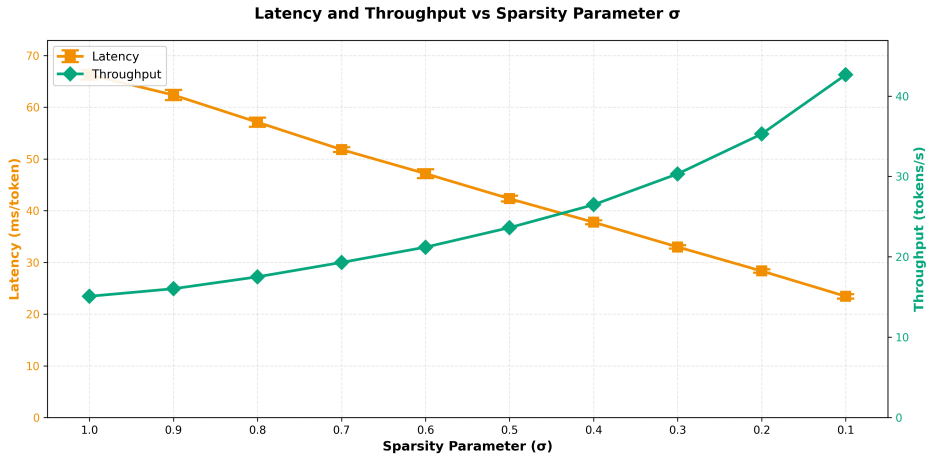


Figure 5.1. Latency (orange, left axis) and throughput (teal, right axis) vs sparsity parameter σ.

Latency decreases monotonically from 66.28 ms at baseline to 23.45 ms at $\sigma = 0.1$, with throughput increasing correspondingly from 15.09 to 42.65 tokens/s (2.83× improvement). Standard deviations remain below 1 ms across all configurations, validating low variance GPU performance. At $\sigma = 0.5$, the 36% latency reduction and 57% throughput increase demonstrate computational savings—the latter translating directly to 57% more concurrent requests per GPU.

Speedup Relative to Theoretical Predictions. Figure 5.2 compares observed speedup against theoretical predictions from Section 2.3.

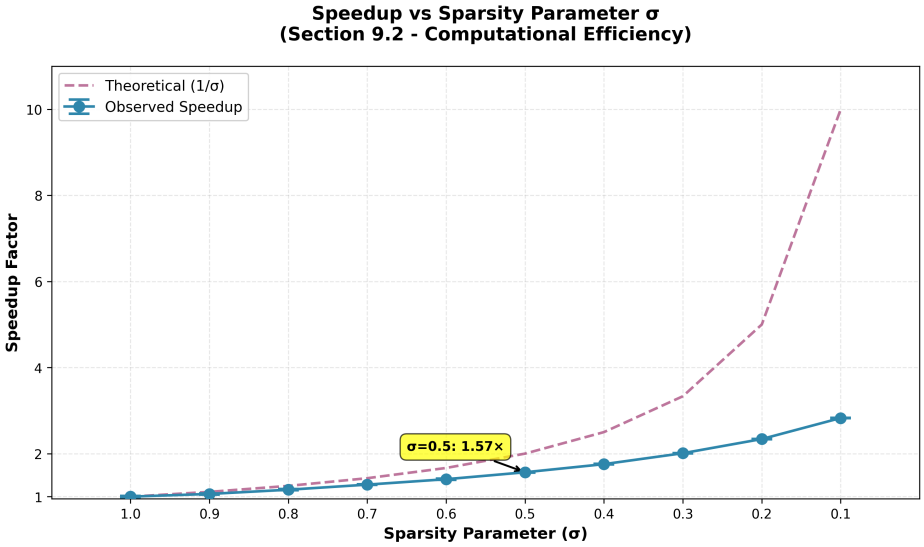


Figure 5.2. Observed speedup (solid blue) vs theoretical $1/\sigma$ prediction (dashed magenta).

The observed speedup curve consistently falls below theoretical predictions, with the gap widening as sparsity increases. At $\sigma = 0.5$, observed speedup of 1.57× compared to theoretical 2.0× represents a 21.5% performance gap; at $\sigma = 0.1$, observed 2.83× falls short of theoretical 10× (71.7% gap). This discrepancy reflects the computational cost of the runtime selection mechanism, which introduces roughly constant additive overhead that becomes proportionally larger as sparse multiplication time decreases.

5.2. Selection Mechanism Overhead

This subsection quantifies the computational cost of the runtime selection mechanism using kernel-level profiling metrics defined in Section 3.2.2, measured on a 4096 × 4096 attention projection matrix.

Kernel-Level Timing Breakdown. Table 5.1 presents absolute execution time for each kernel in the sparse pipeline. Figure 5.3 visualizes the composition of total execution time.

Table 5.1. Per-kernel timing breakdown for 4096 × 4096 matrix multiplication (μs)

σ	Threshold (μs)	Dispatch (μs)	Multiply (μs)	Integrate (μs)	Total (μs)
1.0	22.05	13.42	625.27	8.43	669.17
0.9	22.10	13.44	578.85	8.20	622.59
0.8	22.14	13.36	520.58	7.89	563.97
0.7	21.64	13.39	458.76	7.69	501.47
0.6	21.94	13.32	398.58	7.45	441.29
0.5	21.52	13.32	337.10	7.27	379.20
0.4	20.95	13.29	277.37	7.06	318.67
0.3	20.86	13.23	215.10	6.82	256.01
0.2	21.39	13.21	154.00	6.57	195.17
0.1	21.20	13.10	93.57	6.30	134.16

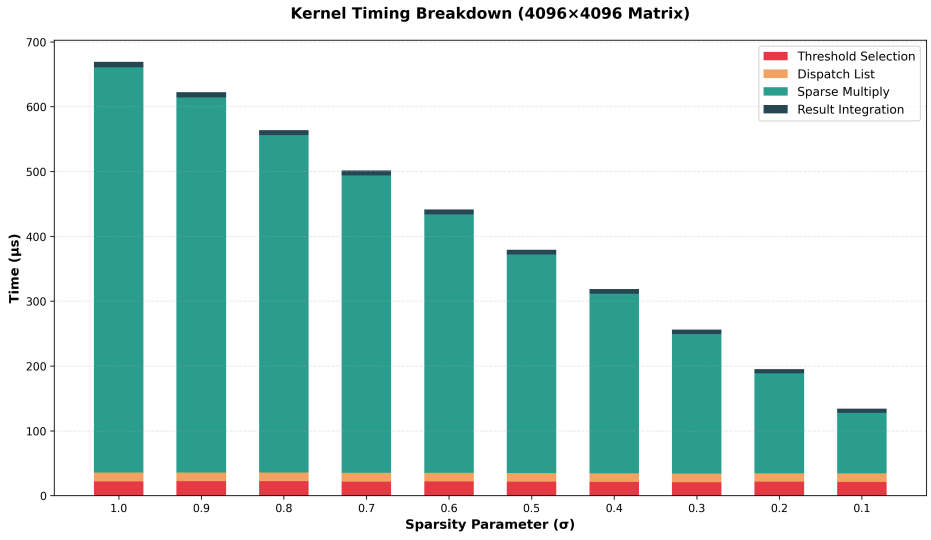


Figure 5.3. Stacked kernel timing breakdown by sparsity level.

The timing breakdown validates theoretical complexity predictions. Threshold selection maintains stable execution time (21 μs) across all sparsity levels, confirming $O(n)$ complexity independent of how many dimensions are selected. Similarly, dispatch list construction exhibits constant time (13 μs). In contrast, sparse multiplication scales linearly with σ: from 625 μs at baseline to 94 μs at $\sigma = 0.1$ (6.7× reduction).

Selection Overhead Growth. Figure 5.4 illustrates the inverse relationship between multiplication time and overhead percentage.

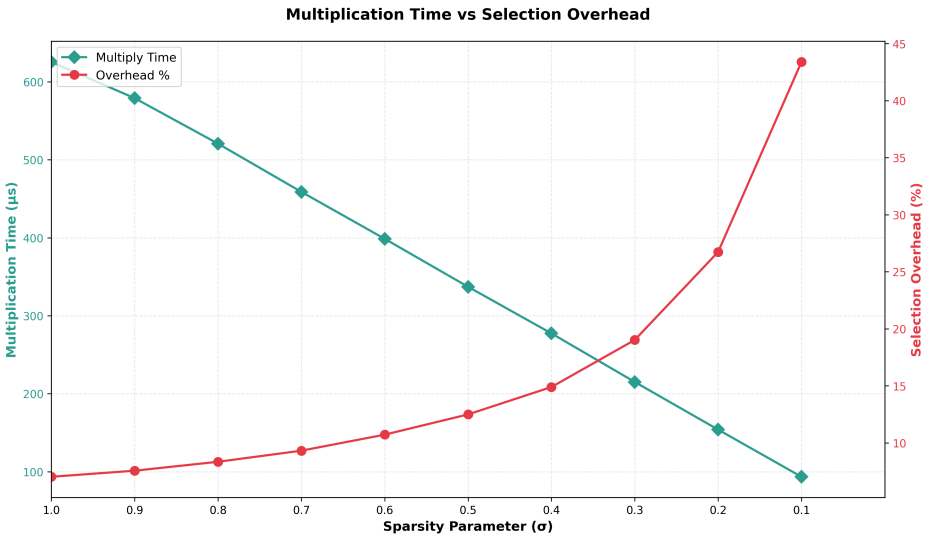


Figure 5.4. Multiplication time (left axis) and selection overhead percentage (right axis) vs σ.

Selection overhead remains approximately constant at 42 μs across all sparsity levels, but its percentage contribution increases as multiplication time decreases. At $\sigma = 0.5$, the 42 μs overhead constitutes 12.5% of total time. Even at baseline ($\sigma = 1.0$), the sparse pipeline incurs 7.0% overhead compared to a hypothetical zero-overhead dense implementation—this fixed cost establishes the performance floor that must be overcome through sparsification gains.

The overhead percentage grows gradually from 7.0% to 12.5% in the range $\sigma \in [0.5, 1.0]$ but accelerates below $\sigma = 0.3$, reaching 43.4% at $\sigma = 0.1$. This exponential growth in relative overhead explains the speedup saturation observed in Section 5.1: while multiplication time at $\sigma = 0.1$ is reduced by 6.7×, overall speedup is limited to 2.83× because nearly half the execution time is consumed by fixed-cost selection operations.

5.3. Output Quality Degradation

This subsection evaluates output quality using perplexity, KL divergence, and token agreement metrics defined in Section 3.2.3, along with factual correctness measured via MMLU accuracy. All measurements are performed on the evaluation datasets described in Section 3.2.6.

Perplexity Degradation. Figure 5.5 presents perplexity measurements across all sparsity levels.

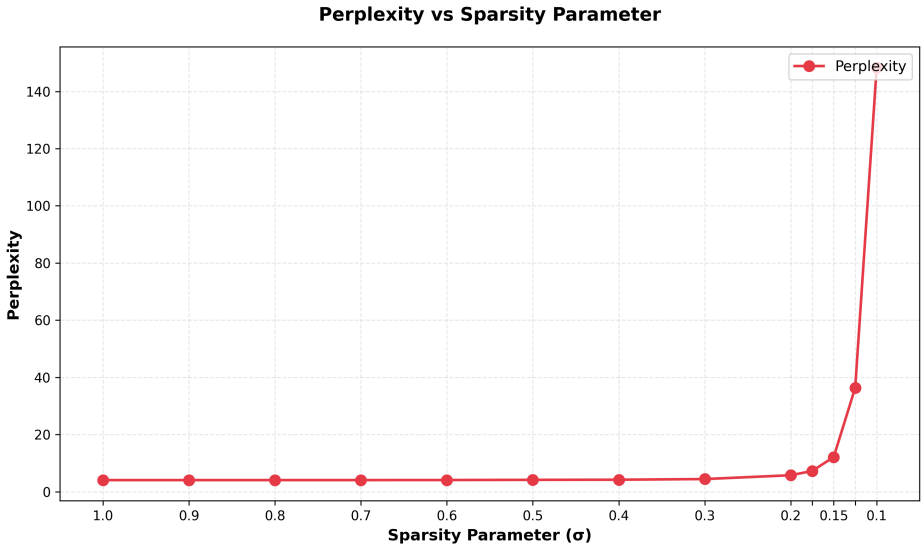


Figure 5.5. Perplexity vs sparsity parameter σ.

Perplexity remains stable for $\sigma \geq 0.5$, increasing by only 2.19% when half of dimensions are excluded. Degradation remains moderate down to $\sigma = 0.3$, with increases of 3.28% at $\sigma = 0.4$ and 8.95% at $\sigma = 0.3$. Below $\sigma = 0.3$, degradation accelerates sharply: 41.62% at $\sigma = 0.2$, 197% at $\sigma = 0.15$, and 3527% at $\sigma = 0.1$. The transition between $\sigma = 0.3$ and $\sigma = 0.2$ marks the onset of model collapse, indicating that approximately 30% of dimensions carry information essential for coherent prediction.

Distributional Shift and Prediction Consistency. Figure 5.6 visualizes both metrics, showing their inverse relationship.

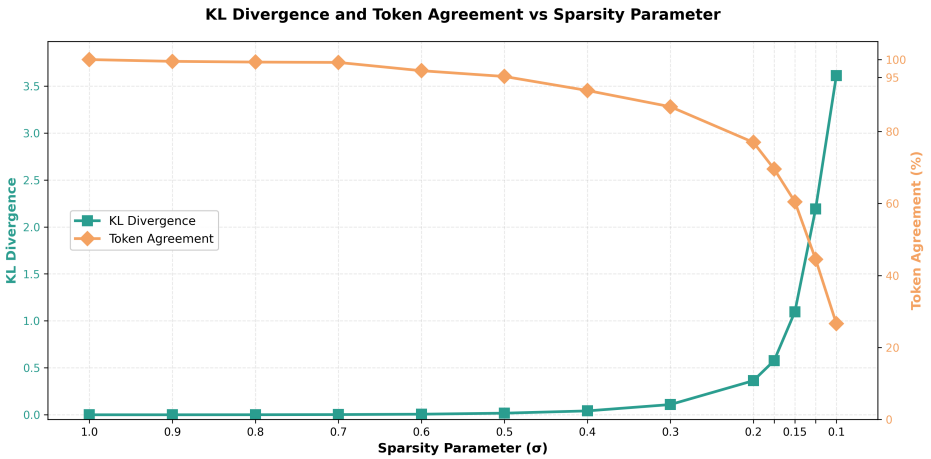


Figure 5.6. KL divergence (teal, left axis) and token agreement (orange, right axis) vs sparsity parameter σ.

Both metrics confirm perplexity findings. At $\sigma = 0.5$, KL divergence of 0.0171 indicates minimal distributional shift, while 95.3% token agreement means the sparse model selects the same top-1 token as baseline in most cases. This reveals that sparsification primarily reduces confidence while maintaining distribution shape—the model becomes less certain but still selects the same tokens. Below $\sigma = 0.3$, both metrics collapse: at $\sigma = 0.1$, KL divergence reaches 3.61 and token agreement drops to 26.6% (near random guessing), confirming model collapse.

Factual Correctness via MMLU. Figures 5.7 and 5.8 present accuracy on 200 MMLU questions across four subjects.

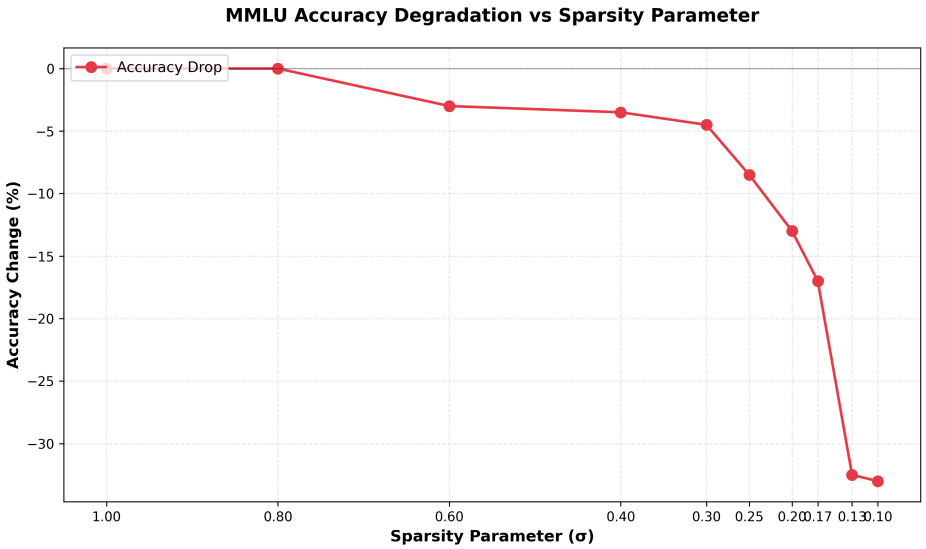


Figure 5.7. MMLU overall accuracy vs sparsity parameter σ. Dashed line indicates random guessing (25%).

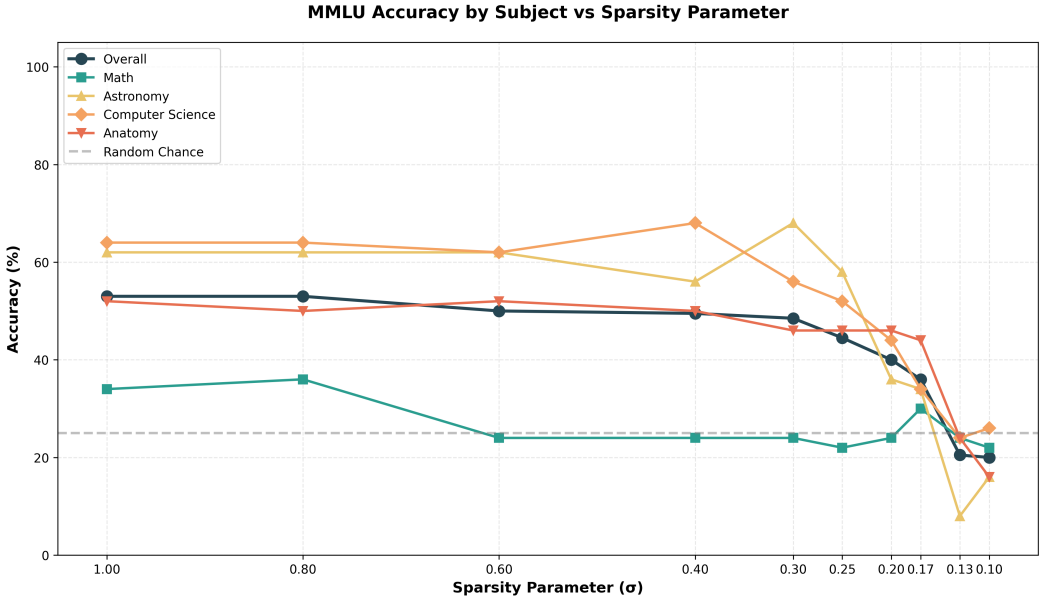


Figure 5.8. MMLU accuracy by subject vs sparsity parameter σ.

MMLU accuracy validates that statistical metrics translate to factual correctness preservation. Accuracy remains stable at 49.5–53.0% for $\sigma \geq 0.4$, representing only 3.5% absolute degradation despite excluding up to 60% of dimensions. This confirms the model retains factual knowledge when operating in the efficient regime. Subject-specific performance shows uniform degradation across domains, indicating that sparsification affects general knowledge retrieval rather than compromising specific reasoning types.

Below $\sigma = 0.25$, accuracy collapses: 40.0% at $\sigma = 0.2$ and 20.0% at $\sigma = 0.1$, the latter indistinguishable from random guessing. The transition between $\sigma = 0.2$ (40%) and $\sigma = 0.17$ (36%) marks catastrophic degradation onset, paralleling perplexity collapse. All subjects converge to near-random performance at $\sigma = 0.1$, confirming uniform model collapse across the entire knowledge base.

5.4. Baseline Comparison

To validate that the sparse pipeline produces correct outputs when all dimensions are selected, we compare $\sigma = 1.0$ (sparse implementation with full selection) against the standard dense Mistral 7B implementation using PyTorch with CUDA. Both implementations are evaluated on identical datasets: 1,000 tokens from WikiText-2 for perplexity and 200 MMLU questions for factual correctness.

Table 5.2 presents the comparison across quality and efficiency metrics.

Table 5.2. Pipeline validation: C++ OpenCL sparse implementation ($\sigma = 1.0$) vs. PyTorch CUDA dense baseline

Metric	**PyTorch Dense**	**C++ σ=1.0**	**Difference**
Quality Metrics			
Perplexity (WikiText-2)	4.097	4.090	-0.17%
MMLU Overall Accuracy	54.33%	53.0%	-1.3%
Mathematics	38.6%	34.0%	-4.6%
Astronomy	57.9%	62.0%	+4.1%
Computer Science	66.0%	64.0%	-2.0%
Anatomy	54.8%	52.0%	-2.8%
Efficiency Metrics			
Mean Latency (ms)	50.97	66.28	+30%
Throughput (tokens/s)	19.62	15.09	-23%

Quality Validation. The perplexity difference of 0.17% validates numerical correctness: despite architectural differences between PyTorch CUDA dense operations and the C++ OpenCL sparse pipeline, both implementations produce statistically indistinguishable predictions when all dimensions are selected. The negligible deviation is attributable to floating-point rounding differences accumulated across 32 layers.

MMLU accuracy shows a 1.3% overall difference (54.33% vs 53.0%), within expected variance for a 200-question benchmark. With baseline accuracy near 50%, the binomial standard error is approximately 3.5%, making the observed difference statistically insignificant. The mixed subject-level results (some higher, some lower) confirm that differences stem from sampling variability rather than implementation errors.

This validation demonstrates that the sparse pipeline preserves model behavior at $\sigma = 1.0$, ensuring that quality degradation at $\sigma < 1.0$ is attributable to dimension exclusion

by the importance-based selection mechanism, not to numerical instability in the sparse infrastructure.

Efficiency Characteristics. The C++ OpenCL implementation exhibits 30% higher latency (66.28 ms vs. 50.97 ms) compared to PyTorch CUDA at $\sigma = 1.0$. This gap reflects two factors: the sparse pipeline's fixed overhead costs (42 μs per matrix operation and the optimization difference between PyTorch's cuBLAS/cuDNN kernels and custom OpenCL implementations.

This baseline overhead establishes that the sparse implementation must achieve sufficient speedup at $\sigma < 1.0$ to surpass PyTorch's dense performance. From Section 5.1, the sparse implementation achieves 42.32 ms latency at $\sigma = 0.5$, which is 17% faster than PyTorch's 50.97 ms baseline. This demonstrates that selective sparsification overcomes the architectural overhead penalty for $\sigma \lesssim 0.6$, achieving faster-than-dense performance despite the sparse pipeline's fixed costs.

5.5. Synthesis of Experimental Findings

This subsection synthesises the experimental results to answer the research questions posed at the beginning of this section.

Computational Efficiency. Selective sparsification achieves measurable speedup scaling with the sparsity parameter σ. Latency decreases monotonically from 66.28 ms at baseline to 23.45 ms at $\sigma = 0.1$, corresponding to throughput increase from 15.09 to 42.65 tokens/s (2.83× improvement). Scaling is sub-linear relative to theoretical predictions: observed 1.57× at $\sigma = 0.5$ versus theoretical 2.0× represents 78.5% efficiency, with the gap attributable to fixed selection overhead.

Selection Overhead. The selection mechanism introduces approximately 42 μs of near-constant overhead per matrix operation, comprising threshold selection (21 μs), dispatch construction (13 μs), and integration (8 μs). This overhead is independent of σ, but its relative contribution grows as multiplication time decreases: from 7.0% at $\sigma = 1.0$ to 12.5% at $\sigma = 0.5$ to 43.4% at $\sigma = 0.1$. The transition at $\sigma \approx 0.3$, where overhead exceeds 19%, marks the onset of diminishing returns from further sparsification.

Output Quality. Quality degrades gradually until $\sigma \approx 0.3$, then collapses. At $\sigma = 0.5$, degradation is negligible: perplexity increases by 2.19%, token agreement remains at 95.3%, and MMLU accuracy falls within sampling variance of baseline. At $\sigma = 0.3$, degradation remains acceptable: perplexity +8.95%, token agreement 86.9%, MMLU 48.5% (4.5 points below baseline). Below $\sigma = 0.3$, degradation accelerates sharply: perplexity increases by 41.62% at $\sigma = 0.2$ and by 3527% at $\sigma = 0.1$, where MMLU accuracy reaches

random-guessing level (20%). The practical lower bound is $\sigma = 0.3$ for applications tolerating 10% quality loss, or $\sigma = 0.5$ for quality-critical deployment.

Baseline Comparison. At $\sigma = 1.0$ (all dimensions selected), the sparse pipeline produces outputs statistically identical to the PyTorch CUDA dense baseline: perplexity difference of 0.17%, MMLU difference of 1.3 percentage points (within sampling variance). The sparse implementation incurs 30% higher baseline latency (66.28 ms vs 50.97 ms) due to selection overhead and the CLBlast-cuBLAS performance gap. However, at $\sigma \leq 0.6$, sparse inference becomes faster than the dense baseline, achieving 17% latency advantage at $\sigma = 0.5$ (42.32 ms vs 50.97 ms).

5.6. Quality-Efficiency Trade-off

The experimental data reveals two thresholds that define the practical operating range for selective sparsification.

Critical Thresholds. The first threshold occurs at $\sigma \approx 0.6$, where sparse inference latency crosses below the dense PyTorch baseline. Above this point, the selection overhead outweighs computational savings; below it, sparsification provides net benefit. The second threshold occurs at $\sigma \approx 0.3$, where quality degradation transitions from gradual to rapid degradation. Between $\sigma = 0.5$ and $\sigma = 0.3$, perplexity increase grows from 2.19% to 8.95%. Between $\sigma = 0.3$ and $\sigma = 0.2$, perplexity increase jumps from 8.95% to 41.62%. The practical operating range is therefore $\sigma \in [0.3, 0.6]$, with the specific choice depending on application requirements.

Recommended Operating Point. For most applications, moderate sparsity (σ around 0.5) provides the balance between speedup and quality preservation. At this level, quality degradation remains negligible across all metrics while delivering throughput improvement. Applications with stricter latency requirements can push to lower σ values (around 0.4) with a bigger quality trade-offs, primarily visible in reduced token agreement with the baseline. Below $\sigma = 0.3$, quality degradation becomes impractical for production use, with factual accuracy approaching random-guessing levels at higher sparsity levels.

Runtime Adaptability. Unlike static pruning methods where the quality-efficiency trade-off is fixed at compression time, selective sparsification allows σ adjustment per-request without model reloading. This enables practical deployment strategies: route quality-sensitive requests (e.g., customer-facing generation) to higher σ values while using lower σ for latency-sensitive or approximate-tolerant requests (e.g., autocomplete, internal tools). Systems under high load could dynamically lower σ to maintain throughput, then restore quality when load decreases.

5.7. Comparison to Related Work

Recent advances in sparse LLM inference provide context for evaluating selective sparsification. We compare against two concurrent works targeting similar sparsity levels: SpInfer [33] and MACKO [34]. Because these methods operate on different hardware (NVIDIA RTX 4090 with CUDA and Tensor Cores versus AMD RDNA3 with OpenCL), different models (OPT-13B and Llama2-7B versus Mistral 7B), and different software stacks, absolute metrics such as latency in milliseconds or throughput in tokens per second are not directly comparable. Instead, we compare *relative speedup*: the ratio of inference time at baseline density to inference time at reduced density, measured against each method's own dense baseline.

Relative Speedup Comparison. Table 5.3 presents relative speedups at approximately 50% effective sparsity. SpInfer reports 1.66× kernel-level speedup over cuBLAS at 50% static sparsity, achieved through Tensor-Core-Aware Bitmap Encoding that exploits NVIDIA's sparse matrix acceleration primitives. MACKO reports 1.2–1.5× speedup over cuBLAS at 50% sparsity using compressed coordinate encoding optimised for sparse matrix-vector multiplication. Our selective sparsification achieves 1.57× speedup at $\sigma = 0.5$ relative to the $\sigma = 1.0$ baseline.

Table 5.3. Relative speedup at approximately 50% sparsity. All values represent speedup over each method's own dense baseline.

Method	Relative Speedup	Hardware	Model
SpInfer	1.66×	RTX 4090 / Tensor Cores	OPT-13B
Selective Sparsification	1.57×	RX 7900 XTX / OpenCL	Mistral 7B
MACKO	1.2–1.5×	RTX 4090 / CUDA Cores	Llama2-7B

Hardware and Model Context. The RTX 4090 used by SpInfer and MACKO offers approximately 82.6 TFLOPS FP16 throughput with dedicated sparse Tensor Core support, compared to 61 TFLOPS on the RX 7900 XTX without equivalent sparse acceleration primitives. OPT-13B (13 billion parameters) is roughly 1.9× larger than Mistral 7B (7.3 billion parameters), with proportionally larger weight matrices that may amortise fixed overheads more favourably. These differences preclude direct latency comparison but do not invalidate relative speedup comparison, since each method is measured against its own baseline under consistent conditions.

Static versus Dynamic Sparsity. The distinction lies in sparsity type. SpInfer and MACKO operate on statically pruned models where weights are permanently removed using algorithms such as SparseGPT or Wanda prior to deployment. The sparsity pattern is fixed

at compression time and cannot be adjusted without re-pruning. Selective sparsification implements dynamic sparsity: all weights are preserved, with per-token selection determining which dimensions participate in each computation based on prediction magnitudes.

This distinction has practical implications. Static methods achieve their speedup with zero per-token overhead—the sparse structure is precomputed and encoded in the weight representation. Selective sparsification incurs approximately 42 μs of selection overhead per matrix operation, which partially offsets computational savings. However, dynamic selection enables capabilities that static methods cannot provide: the sparsity parameter σ can be adjusted per-request without model reloading, different inputs naturally use different weight subsets based on their prediction patterns, and quality degrades gracefully as σ decreases rather than exhibiting the threshold effects associated with aggressive static pruning.

Complementary Positioning. For deployment scenarios where throughput must be maximised and quality requirements are fixed in advance, static pruning may be preferable—the absence of selection overhead allows translation of sparsity into speedup. For applications requiring runtime quality-efficiency trade-offs, serving heterogeneous request types with different latency requirements, or preserving the option to restore full model capacity, selective sparsification offers flexibility that static methods cannot match. The approaches are compositionally compatible: a model could be statically pruned to remove consistently unimportant weights, then selectively sparsified at inference time for additional per-token adaptation.

5.8. Analysis of the Speedup Gap

The experimental results reveal a consistent gap between theoretical and observed speedup. At $\sigma = 0.5$, theoretical 2.0× reduces to 1.57× in practice. At $\sigma = 0.1$, theoretical 10× becomes 2.83×.

Selection Overhead. The selection mechanism introduces approximately 42 μs of near constant overhead per matrix operation, comprising threshold selection (21 μs), dispatch construc- tion (13 μs), and integration (8 μs). This overhead is independent of σ, but its relative contribution grows as multiplication time decreases. At $\sigma = 1.0$, the 35 μs overhead is amortised across 625 μs multiplication time (5.6% overhead). At $\sigma = 0.1$, the same overhead is amortised across only 62.5 μs theoretical multiplication time (56% overhead). This constant-overhead effect explains the sub-linear speedup curve but does not fully account for the observed gap.

The selection mechanism introduces approximately 42 μs of near-constant overhead per matrix operation, comprising threshold selection (21 μs), dispatch construc- tion (13 μs), and integration (8 μs).

Architectural Cause: Wavefront Memory Coalescing. The AMD RDNA3 architecture organises compute threads into wavefronts of 64 threads that execute in lockstep. Peak memory bandwidth is achieved only when threads within a wavefront access contiguous memory addresses, allowing the memory controller to coalesce multiple requests into wide transactions. Dense matrix multiplication exhibits predictable, strided access patterns where consecutive threads fetch consecutive weight columns, enabling full coalescing.

Selective sparsification produces irregular, data-dependent access patterns. The indices of retained dimensions are determined by prediction magnitudes and vary per token. For example, at $\sigma = 0.5$, a typical selection might retain dimensions $\{0, 5, 12, 19, 23, 47, ...\}$ rather than contiguous $\{0, 1, 2, 3, 4, 5, ...\}$. When 64 threads simultaneously fetch weight columns at these non-contiguous indices, the memory controller cannot coalesce the requests—each thread generates a separate memory transaction. At $\sigma = 1.0$, consecutive threads access consecutive columns (perfect coalescing). At $\sigma = 0.5$, average distance between selected dimensions is approximately 2, reducing coalescing efficiency. Increasing sparsity further worsen fragmenting memory access.

Optimisation Targets. Rather than selecting individual dimensions based on magnitude, selecting contiguous blocks of dimensions (e.g., 8 or 16 consecutive indices) would restore partial coalescing at the cost of selection granularity. A hybrid approach might use fine-grained selection at moderate sparsity ($\sigma \geq 0.5$) where bandwidth degradation is mild, switching to block-structured selection at aggressive sparsity where coalescing becomes critical.

6. Conclusion

This thesis presented input-adaptive selective sparsification, a method for reducing large language model inference cost through dynamic, per-token decisions about which computations to perform. Rather than permanently removing weights as in static pruning, the method preserves full model capacity and determines at runtime which dimensions of matrix operations contribute meaningfully to the output for each specific input. The experimental evaluation demonstrated that selective sparsification achieves measurable speedup with minimal quality degradation at moderate sparsity levels, and outperforms dense inference when the sparsity parameter is less than 0.6. The key advantage is runtime flexibility: the quality-efficiency trade-off can be adjusted per-request without model reloading, enabling deployment strategies that adapt to varying requirements and system load.

Limitations. All experiments were conducted on a single model (Mistral 7B) and hardware platform (AMD GPU with OpenCL). While Mistral 7B is representative of modern LLM architectures, the results may not generalise to larger models (70B+ parameters) or Mixture-of-Experts architectures. Larger models may exhibit different importance concentration patterns, potentially favouring more or less aggressive sparsification, and the fixed selection overhead would constitute a smaller fraction of total computation in proportionally larger MLP layers. MoE architectures present a distinct question: expert routing and dimension selection could yield compounding efficiency gains, or introduce interference if gating decisions conflict with importance-based selection. Porting to NVIDIA GPUs with CUDA could yield improved performance through more optimized tooling and hardware acceleration primitives not available on the current platform.

Future Work. Two directions extend the core approach. First, memory access patterns present the largest opportunity for performance improvement. Section 5.8 identified irregular memory access as a source of efficiency loss at aggressive sparsity, with non-contiguous dimension selection preventing effective memory coalescing. Block-structured selection, where contiguous groups of dimensions are selected together, would restore memory coalescing at the cost of selection granularity. A hybrid approach using fine-grained selection at moderate sparsity and block-structured selection at higher sparsity could capture benefits of both strategies. Second, the principle of input-adaptive computation could operate at coarser granularity through dynamic layer skipping, where entire transformer layers are bypassed when their predicted contribution falls below a threshold. Combining dimension-level sparsification with layer-level skipping would provide hierarchical computation allocation. Additional engineering work includes characterizing speedup scaling across batch sizes and evaluating interaction with quantization, where combined benefits may be multiplicative but error accumulation requires analysis.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need", in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners", *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, *et al.*, "Language models are few-shot learners", in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.

[4] A. Chowdhery, S. Narang, J. Devlin, *et al.*, "Palm: Scaling language modeling with pathways", *arXiv preprint arXiv:2204.02311*, 2022.

[5] H. Touvron, L. Martin, K. Stone, *et al.*, "Llama 2: Open foundation and fine-tuned chat models", *arXiv preprint arXiv:2307.09288*, 2023.

[6] A. Q. Jiang, A. Sablayrolles, A. Mensch, *et al.*, "Mistral 7b", *arXiv preprint arXiv:2310.06825*, 2023.

[7] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, *et al.*, "Scaling laws for neural language models", *arXiv preprint arXiv:2001.08361*, 2020.

[8] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, *et al.*, "Training compute-optimal large language models", *arXiv preprint arXiv:2203.15556*, 2022.

[9] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, "Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks", *Journal of Machine Learning Research*, vol. 22, no. 241, pp. 1–124, 2021.

[10] T. Gale, E. Elsen, and S. Hooker, "The state of sparsity in deep neural networks", *arXiv preprint arXiv:1902.09574*, 2019.

[11] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural network", in *Advances in Neural Information Processing Systems*, 2015.

[12] Y. Guo, A. Yao, and Y. Chen, "Dynamic network surgery for efficient dnns", in *Advances in Neural Information Processing Systems*, vol. 29, 2016.

[13] B. Jacob, S. Kligys, B. Chen, *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference", *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2704–2713, 2018.

[14] T. N. Sainath, B. Kingsbury, V. Sindhwani, E. Arisoy, and B. Ramabhadran, "Low-rank matrix factorization for deep neural network training with high-dimensional output targets", in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2013, pp. 6655–6659.

[15] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network", *arXiv preprint arXiv:1503.02531*, 2015.

[16] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks", in *23rd International Conference on Pattern Recognition (ICPR)*, 2016, pp. 2464–2469.

[17] N. Shazeer, A. Mirhoseini, K. Maziarz, *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer", in *International Conference on Learning Representations*, 2017.

[18] A. Graves, "Adaptive computation time for recurrent neural networks", in *arXiv preprint arXiv:1603.08983*, 2016.

[19] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, *et al.*, "Transformers: State-of-the-art natural language processing", in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.

[20] G. Penedo, Q. Malartic, D. Hesslow, R. Cojocaru, *et al.*, "The refinedweb dataset for falcon llm: Outperforming curated corpora with web data, and web data only", *arXiv preprint arXiv:2306.01116*, 2023.

[21] G. Team *et al.*, "Gemma: Open models based on gemini research and technology", *arXiv preprint arXiv:2403.08295*, 2024.

[22] DeepSeek-AI, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning", *arXiv preprint arXiv:2501.12948*, 2025.

[23] A. Yang, B. Yang, B. Hui, *et al.*, "Qwen2 technical report", *arXiv preprint arXiv:2407.10671*, 2024.

[24] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding", *arXiv preprint arXiv:2104.09864*, 2021.

[25] N. Shazeer, "Glu variants improve transformer", *arXiv preprint arXiv:2002.05202*, 2020.

[26] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints", in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[27] F. Jelinek, R. L. Mercer, L. R. Bahl, and J. K. Baker, "Perplexity—a measure of the difficulty of speech recognition tasks", *The Journal of the Acoustical Society of America*, vol. 62, no. S1, S63, 1977.

[28] S. F. Chen and J. Goodman, "An empirical study of smoothing techniques for language modeling", *Computer Speech & Language*, vol. 13, no. 4, pp. 359–394, 1999.

[29] S. Kullback and R. A. Leibler, "On information and sufficiency", *The Annals of Mathematical Statistics*, vol. 22, no. 1, pp. 79–86, 1951.

[30] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter", *arXiv preprint arXiv:1910.01108*, 2019.

[31] D. Hendrycks, C. Burns, S. Basart, *et al.*, "Measuring massive multitask language understanding", *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[32] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models", *arXiv preprint arXiv:1609.07843*, 2016.

[33] R. Fan, X. Yu, P. Dong, *et al.*, “SpInfer: Leveraging low-level sparsity for efficient large language model inference on GPUs”, in *Twentieth European Conference on Computer Systems (EuroSys ’25)*, ACM, 2025.

[34] V. Macko and V. Boža, “MACKO: Sparse matrix-vector multiplication for low sparsity”, *arXiv preprint arXiv:2511.13061*, 2025.